

TRAVELING SALESPERSON PROBLEM

Aleksey Sokolov Matr.: 12004091

Monte Carlo Methods
Supervisor: Sexty, Denes, Assoz. Prof. PhD
31. August 2026

Inhaltsverzeichnis

1	Introduction	3
2	fundamentals	3
2.1	Computational Complexity and Algorithmic Approaches	3
2.2	Physical and Industrial Applications	3
2.3	Theoretical Significance	4
2.4	Timeseries Analysis	4
2.4.1	Autocorrelation and effective sample size	4
2.4.2	Blocking	4
3	Methods	4
3.1	Simulated Annealing	5
3.2	Combined Algorithm (Genetic + Annealing)	6
4	Results	8
4.1	Validation of Monte Carlo Sampler	8
4.1.1	Metropolis-Hastings Algorithm (Annealing)	11
4.1.2	Combined Algorithm	14
4.2	Autocorrelation	16
4.2.1	Metropolis-Hastings Algorithm (Annealing)	17
4.2.2	Combined Algorithm	18
4.3	Benchmarking	18
4.3.1	ch150	18
4.3.2	dsj1000 - temperature cycling	21
5	Conclusion	25

1 Introduction

The traveling salesman problem is a popular entry problem for computational optimization using probabilistic methods. It was first formulated in 1930 and is since then used as a benchmark for many optimization algorithms. Even though the problem is computationally difficult, many heuristics and exact algorithms are known, which can approximate the exact solution within a small fraction of 1 %. This document focuses on implementing two different approaches to finding the optimal path and then comparing their runtime and uncertainties their calculations. First we focus on implementing a simple Metropolis algorithm with simulated annealing, where we compare different annealing strategies with the use of temperature functions. The second part of this document a genetic algorithm is implemented and at last compared to the probabilistic approach.

2 fundamentals

2.1 Computational Complexity and Algorithmic Approaches

The factorial scaling of the symmetric TSP, given by $(N - 1)!/2$ distinct routes, renders brute-force methods impractical beyond small system sizes. Exact algorithms, such as the *Held-Karp dynamic programming approach*, provide solutions but require exponential computational resources, limiting their applicability. As a result, most practical implementations rely on *heuristic and approximation algorithms*, which yield near-optimal solutions at significantly reduced computational cost.

Among the most effective heuristic methods are:

- **Greedy algorithms**, which construct solutions iteratively based on local distance minimization.
- **Metaheuristic approaches**, including *simulated annealing*, *genetic algorithms*, and *ant colony optimization*, which employ stochastic strategies to escape local minima.
- **Mathematical optimization techniques**, such as *integer linear programming relaxations* and *cutting-plane methods*, which refine feasible regions iteratively.

Although these methods do not guarantee exact solutions, they remain essential in real-world applications where computational constraints necessitate efficient approximations.

2.2 Physical and Industrial Applications

Optimization problems analogous to the TSP emerge in numerous domains, particularly in physics and engineering. Examples include:

- **Path optimization in transport networks**, where fuel efficiency and travel time minimization are critical.
- **Manufacturing processes**, where CNC machining and robotic assembly require minimizing tool movement to enhance efficiency.
- **Computational biology**, where the sequencing of DNA fragments or protein folding pathways presents combinatorial structures comparable to TSP-like models.

Beyond applied optimization, the TSP is frequently utilized as a *benchmark problem* for evaluating new algorithmic techniques, serving as a standard reference in computational complexity research.

2.3 Theoretical Significance

The TSP holds deep implications in computational complexity theory, particularly in relation to the *P vs NP problem*. As an NP-hard problem, the TSP is unlikely to have a polynomial-time solution unless $P = NP$, a hypothesis that remains one of the most significant open questions in theoretical computer science. A resolution to this problem would not only transform computational mathematics but also have profound consequences for cryptography, artificial intelligence, and algorithmic research.

2.4 Timeseries Analysis

2.4.1 Autocorrelation and effective sample size

Autocorrelation measures the correlation of a signal with a delayed copy of itself.

The analysis of autocorrelation is a mathematical tool for identifying repeating patterns or hidden periodicities within a signal obscured by noise.

In the context of Monte Carlo time series, it is used to measure how strongly successive stationary observables A_n are correlated with each other.

The autocorrelation function for a stationary observable is defined as

$$C_A(t) = \frac{\mathbb{E}[A_n, A_{n+1}] - \mathbb{E}[A]^2}{\sigma_A^2} \quad (1)$$

The integrated autocorrelation time is convention dependent. Using the lecture convention:

$$\tau_{\text{int}} = \frac{1}{2} + \sum_{t=1}^{\infty} C_A(t) \quad (2)$$

For $N \gg \tau_{\text{int}}$ the variance of the observables mean $\bar{A}$ can be defined as:

$$\text{Var}(\bar{A}) = \frac{2\tau_{\text{int}}}{N} \sigma_A^2 \quad (3)$$

2.4.2 Blocking

The ordered series is split into consecutive blocks of length B , each block is replaced by its mean. When $B \gg \tau_{\text{int}}$, block means are approximately independent and the estimated error reaches a plateau. Randomly shuffling first would destroy the time-correlation information.

3 Methods

There are three algorithms used in this project paper. In this section these three methods are explained and discussed.

3.1 Simulated Annealing

Simulated Annealing is a computational method that takes inspiration from solid state physics, namely from solid annealing and crystalization. As solids are heated they form different crystal structures. If left to cool local order emerges, driven by the tendency of a material to reduce internal stress and to attain a lower energy state. The same principle is applied to the a sequence of city positions where the order in which the cities are visited strongly influences the path length. In this approach, the length of sequentially ordered cities is considered the Free Energy of the system:

$$L(\tau) = E(\tau) = \sum_{i=1}^N |\mathbf{x}_{i+1} - \mathbf{x}_i| \quad \text{with} \quad \mathbf{x}_{N+1} = \mathbf{x}_1. \quad (4)$$

In this, the $(N - 1)!/2$ combinations of states (particular permutations) form a statistical ensemble, which corresponds to a partition function. The partition function encapsulates the statistical properties of the system in thermodynamic equilibrium. It serves as a function of thermodynamic state variables, such as temperature, and provides access to macroscopic quantities like free energy through its derivatives. The canonical partition function, commonly used in such contexts, describes a system in thermal equilibrium with its environment at a fixed temperature. Since direct computation of the partition function by summing over all possible states is infeasible due to the factorial growth of possible permutations, Simulated Annealing is employed to explore the configuration space. For the sake of understandability the energy will be replaced by the length, as it is the same quantity in this kind of systems.

$$Z = \sum_i e^{-L(\tau_i)/T} \quad (5)$$

One can use the Boltzman distribution to determine the probability of observing a state τ at a given temperature and observed length.

$$p_L(\tau|T) = \frac{1}{Z} \exp \left\{ -\frac{1}{T} L(\tau) \right\} \quad (6)$$

The expectation value for the length L at a given temperature T is than given by:

$$\langle L \rangle_T = \sum_i L(\tau_i) \cdot p_L(\tau_i|T) \quad (7)$$

Consequently one can also define the variance of L with:

$$\text{Var}(L)_T = \langle L^2 \rangle_T - \langle L \rangle_T^2 \quad (8)$$

From the definition of the variance an analog for the heat capacitance can be defined:

$$C(T) = \frac{\text{Var}(L)_T}{T^2} \quad (9)$$

Since Z cannot be computed in a reasonable time for large systems, this distribution tells that a thermodynamic system favors lower energy states, especially at lower temperatures. This probability can be used to construct an acceptance probability for changing states as seen in eq. 10. By this definition negative Energy changes are always accepted, while positive are

decreasingly unlikely with lower temperature. In combination with temperature variations this allows for efficient exploration of the phase space.

$$\min \left(1, \frac{p_E(\tau_2|T)}{p_E(\tau_1|T)} \right) \quad (10)$$

In order to traverse the energy landscape, a appropriate change must be proposed. This is realized by taking a random subpath of the cities sequence K by generating two random indeces $(a, b) \in (1, N)$ and flipping it, as seen here:

$$K : S_1, \dots, S_{a-1}, \underbrace{S_a, S_{a+1}, \dots, S_{b-1}, S_b}_{old}, S_{b+1}, \dots, S_N \quad (11)$$

$$K' : S_1, \dots, S_{a-1}, \underbrace{S_b, S_{b-1}, \dots, S_{a+1}, S_a}_{new}, S_{b+1}, \dots, S_N \quad (12)$$

The advantage of this proposal is that one can efficiently calculate the change in energy without iterating over the whole array:

$$E(K') - E(K) = \overline{S_{a-1}S_b} + \overline{S_aS_{b+1}} - \overline{S_{a-1}S_a} - \overline{S_bS_{b+1}} \quad (13)$$

By applying the changes and comparing a uniform generated number with the acceptance probability one can minimize the energy. If one chooses a appropriate temperature function, which changes the temperature input every sweep (N^2), local minima can be explored. Even a global minima can be found, but this can not be verified even for relatively few cities.

3.2 Combined Algorithm (Genetic + Annealing)

This approach is derived from nature. (Kind of) It simulates the natural evolution of a population of individual Node sequences. As in nature there are steps of inheritance, mutation, and natural selection. There are several ways to implement this concept, and the used algorithm is a basic version with only the core steps. In this approach the cost function needs to be evaluated very often, and the length needs to be calculated in its entirety. At first a population from one path is created by shuffling the path N times and saving the individuals. This ensures genetic diversity at the start.

The following steps are implemented, which are done each sweep:

- **Selection**

This is done by choosing survivors of a the current population by taking uniform random pairs (a, b) and comparing their lenghts. The population array is shuffled and split in half. This ensures a that not only the best paths are selected and genetic variety is conserved.

$$P : K_1, \dots, K_a, \dots, K_b, \dots, K_M \quad (14)$$

$$E(K_a) > E(K_b) \quad (15)$$

By comparing random paths from the population, there is a chance of some unfit paths to still survive. However the worst path is always dropped.

- **Reproduction**

Pairs of survivors are taken at random and their genome is combined to form a new solution. This is done by taking a random subpath between indecess (a, b) of one parent

and inserting it into the genome of the other parents genome. Each pair of survivors gives rise to two offspring. The following method is applied to the pair $(i \rightarrow j)$ and $(j \rightarrow i)$ of two random individuals. So both can share a subgenom **SG** (subpath) of their whole genom **G** (tour).

$$\begin{aligned} \mathbf{G}_i &: \{\mathbf{ID}_0, \dots, \mathbf{ID}_{42}, \dots, \mathbf{ID}_{11}, \dots, \mathbf{ID}_{142}, \dots, \mathbf{ID}_N\} \\ &\quad \uparrow \text{match} \\ \mathbf{SG}_j &: \{\mathbf{ID}_a, \mathbf{ID}_{142}, \mathbf{ID}_{11}, \mathbf{ID}_{42}, \mathbf{ID}_b\} \end{aligned}$$

The **ID**'s of $\mathbf{SG}_j$ are matched in the genom of $\mathbf{G}_i$, while not matched **ID**'s are stored in a separate set. What remains of $\mathbf{G}_i$ is then stored in an set $\mathbf{RG}_i : \mathbf{G}_i / \mathbf{SG}_j$ (remaining genom). This both subsequences are then used to construct a new genom by inserting the subgenom $\mathbf{SG}_j$ in the same place as it was in the original $\mathbf{G}_j$ (between indices a and b). The remaining genom $\mathbf{RG}_i$ entries are then placed outside of the indices a and b in the same ordering they were in $\mathbf{G}_i$.

- **Mutation**

This step is generally implemented by randomly exchanging ordering. In this algorithm the metropolis-hastings step is used.

Genetic algorithms are generally fast in finding a solution, even if the cost function has a complex shape, because they can traverse the configuration space with a bigger coverage based on the genetic variety. However, it is less efficient in finding the optimum itself. Only the reproduction and the selection part are used. The idea is very simple: to combine the annealing process of simulated annealing with the evolutionary optimization of the genetic algorithm. Simulated annealing focuses on local optimization, making it effective at refining solutions but limited in exploration. On the other hand, the genetic selection and reproduction produces a bigger coverage but does not efficiently find the optimum, as discussed before. The key is to find a good balance between the two. For this the probability of mutation is guided by the Boltzmann distribution, and multiple mutations in a timestep are allowed. Also, this approach introduces the temperature T as an external variable, which can influence the population. One way to think about this is in the evolutionary context. Certain environments can allow a population of a species to mutate, which are less crucial to the survival by pure chance. Like in the case of the dodo, if the environment is not forcing it to have beneficial mutations, birds can lose the ability to fly but gain size. This would be the case for higher temperatures. The temperature in this case can be thought of as evolutionary pressure and not the literal temperature that an individual can experience.

For this algorithm some metrics are introduced additionally to analyze populations. The first metric is the mean edge diversity, which just looks at the shared edges d_{ij} of members of the population. For this, both neighbors of all nodes of all population members are stored and compared between population members. If a node ID in population member i has the same neighbor as the same node ID of population member j , it gets counted.

$$\text{edge diversity} = 1 - \frac{1}{NP^2} \sum_{ij}^{P^2} d_{ij} \quad (16)$$

This should be a sensible estimate for the diversity of a population. Additionally, a second metric is used. Here all subsequences s of length $a < L_s < N_{Nodes}$ are counted and normalized

by a shared subsequence of a completely homogeneous population set. For this replica set the count would be $P^2(N_{Nodes} - a)$. This shall be called likeness S .

$$S = 1 - \frac{1}{P^2(N_{Nodes} - a)} \sum_{ij}^{P^2} s_{a < L_s < N_{Nodes}} \quad (17)$$

These two measures are not only used as the classification of the diversity but also can be interpreted as a distance between state vectors. I don't want to claim that this is a rigorous metric, but it gives an estimate. One weakness is that this "pseudo-metric" is normalized on a population, so the distance is always in respect to the size and the variance between a population. An outlier state vector distance would always be in respect to the diversity of a population or an independent set of state vectors.

4 Results

4.1 Validation of Monte Carlo Sampler

The most interesting direction is to treat a TSP tour as a statistical-mechanical microstate. Let x be a microstate in the $(N - 1)!/2$ dimensional state space. The probability to visit a state x is then given by the formula 6. Now this quantity is generally not known, as the state space cannot be explored fully for even a small number of nodes.

Nevertheless, a system of 11 nodes can still be solved quite quickly, as it has only 1814400 possible unique permutations. The 11 nodes were generated by using the random package from NumPy with the seed 30121999 in the following form.

$$x/y \sim \mathcal{U}_{\mathbb{Z}}(\text{low}, \text{high} - 1) + \mathcal{N}\left(0, \frac{|\text{high}|}{2}\right) \quad (18)$$

All numbers of unique permutations were computed, and the partition function 5, the expectation value 7, and the variance 8 were evaluated in the temperature range (0.05;50) with a resolution of 0.01. This Node configuration will serve as an exactly solved system, which we can use to analyze the sampling of the algorithms presented in section 3. The distribution of the nodes as well as the optimal distance $\min(L(\tau))$ is shown in figure 1.

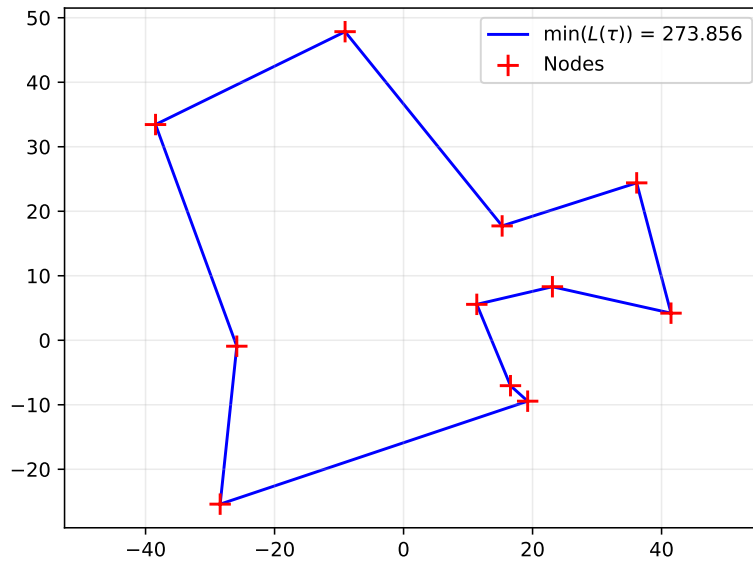


Abbildung 1: Eleven Randomly distributed Nodes according to eq. 18 and optimal distance

As the system is solved fully, we can also calculate the probability of visiting the optimal state for each temperature with formula 6. In figure 2 the probabilities for visiting the minimal-, maximal-, and mean-length state τ were calculated.

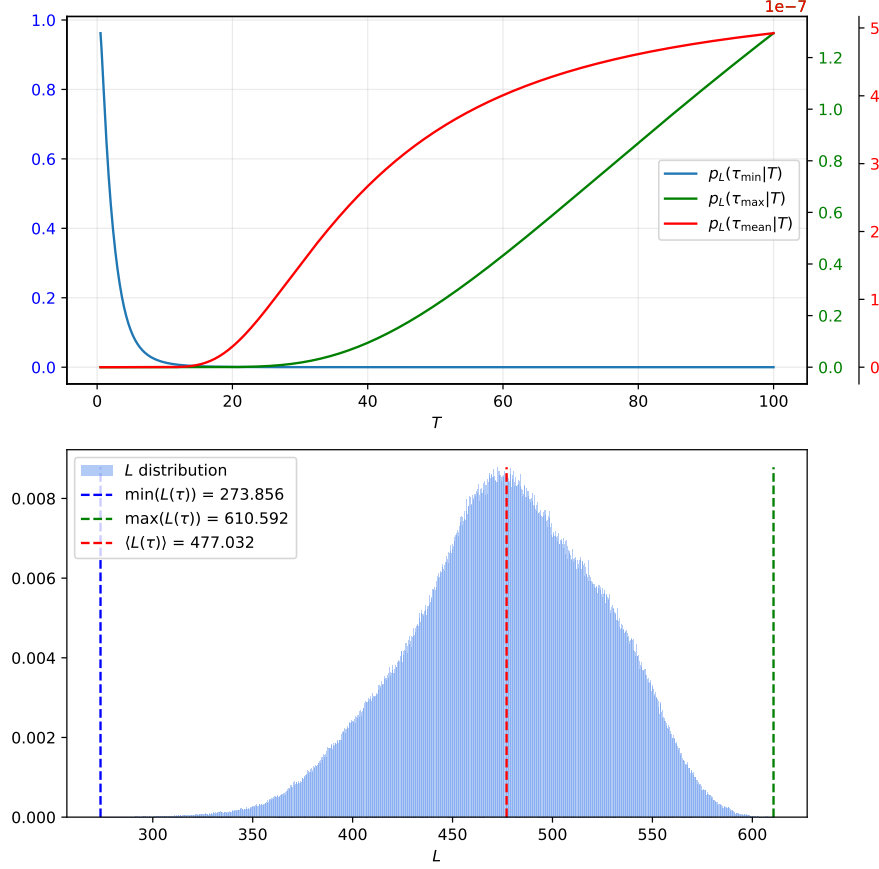


Abbildung 2: top: $p_L(\tau|T)$ for different lengths produced by the Node configuration, bottom: length distribution

It is apparent that visiting the optimal state is most probable at low temperatures as predicted by the equilibrium Boltzmann distribution. As $T \rightarrow 0^+$ only the shortest tour survives. Factoring out the weight of the shortest state from eq. 6 gives:

$$p_L(\tau_{\min}|T) = \frac{1}{1 + \sum_{\tau \neq \tau_{\min}} e^{-(L(\tau) - L(\tau_{\min}))/T}} \quad (19)$$

From eq. 19 follows that every difference $L(\tau) - L(\tau_{\min})$ approaches zero; therefore, the probability as a whole approaches 1. Now this is nice, but what happens in the high-temperature limit?

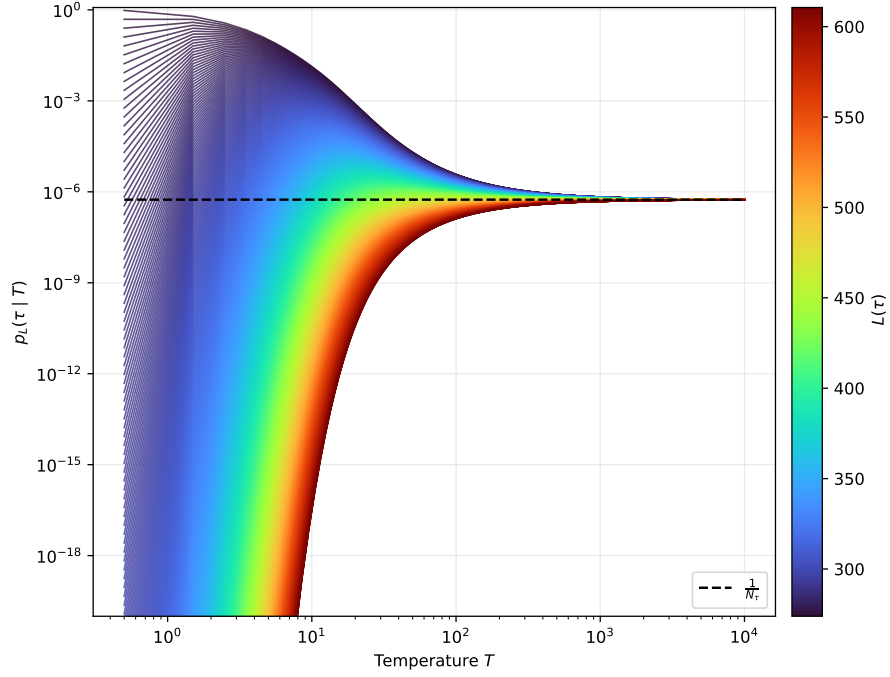


Abbildung 3: probabilities of reaching optimal length

In figure 3 the probabilities for lengths in the range of the lengths distribution from 2 are plotted up to a temperature of 10000. At high temperatures every state change becomes equally probable, this is easily shown when taking the limit $T \rightarrow \infty$ of eq. 6:

$$p_L(\tau_i|T)_{T \rightarrow \infty} = \frac{e^0}{\sum_{N_\tau} e^0} = \frac{1}{N_\tau} \quad (20)$$

4.1.1 Metropolis-Hastings Algorithm (Annealing)

To test if the standard annealing algorithm truly samples the correct Boltzmann distribution, the exact solution was compared to the observables measured in between sweeps at constant temperature. A sweep in this case consists of N_{Nodes}^2 state change proposals. This ensures that the system is equilibrated before measuring. The implemented algorithm performed 8000 sweeps for each temperature in the range 0.05 to 50 with a resolution of 0.01. However, 20 % of the sweeps for each temperature were used for the warmup procedure to ensure that all states can be reached. This is especially important for low temperatures, because the probability for state change is extremely low. The first 2000 sweeps were performed with the temperature function:

$$T(n_{\text{sweep},i}, T_{\text{finish}}) = \frac{1200}{n_{\text{sweep},i} + 1} + T_{\text{finish}} \quad (21)$$

Following the warm-up procedure, the algorithm was run with constant temperature, and after each sweep, the last length as well as the acceptance rate were observed and tracked. This gave a sample set of 6000 for each temperature. The quantities in figure 4 are then calculated using the standard formulas for the mean and variance. The exact solution was calculated using the equations [5,7,8, 9].

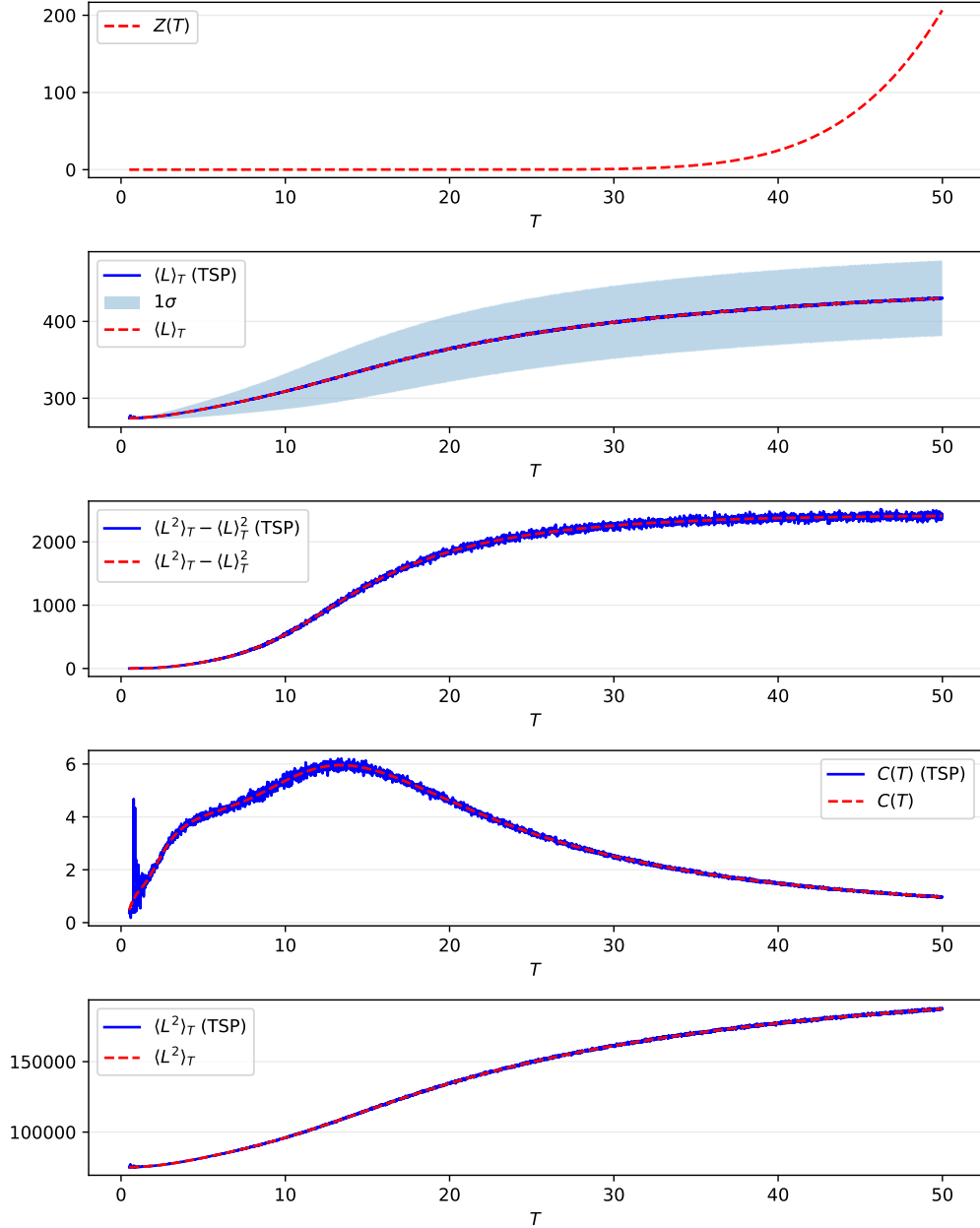


Abbildung 4: Comparison between the exact solution (red) and the sampled observables using the annealing algorithm (blue)

The peak in the heat capacity at approximately $T=13$ indicates the temperature range in which the distribution changes most strongly. At low temperatures, the system mainly occupies short tours, and the fluctuations in length are small. With increasing temperature, longer tours become accessible, and the variance increases rapidly. At high temperatures, the variance approaches a constant value, while $\langle L^2 \rangle_T$ continues to increase, causing the heat capacity to decrease again. The peak therefore marks the temperature range in which the mean tour length changes most strongly with temperature. It can be seen as a crossover between ordered and disordered tours and should not be seen as a real phase change. The mean acceptance rate for each temperature is shown in figure 5.

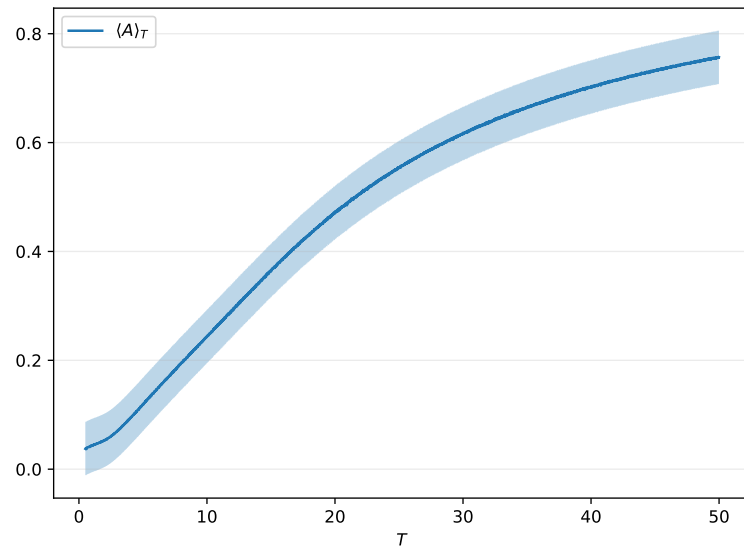


Abbildung 5: Acceptance

4.1.2 Combined Algorithm

The mutation in the combined algorithm is done using the Metropolis-Hastings annealing step, as discussed before. However, the mutation and crossover part of the genetic part breaks detailed balance; therefore, it is not sensible to expect it to sample from the Boltzmann distribution. Nevertheless, the sampling was compared to the exact solution to observe the difference. The evaluation of the combined algorithm was separated into two edge cases, in which it is genetic-dominated and annealing-dominated. In figure 6, the annealing-dominated case is seen with the mutation consisting of N_{Nodes}^2 proposals. One can see especially at higher temperatures the observables are sampled from a Boltzmann-like distribution, but at low temperatures there is a mismatch. This is due to the crossover not being dependent on temperature, while the accepted proposals are. Even though the heat capacity is plotted, it should not be interpreted as one, because the genetic part is stochastic and, as said before, breaks detailed balance.

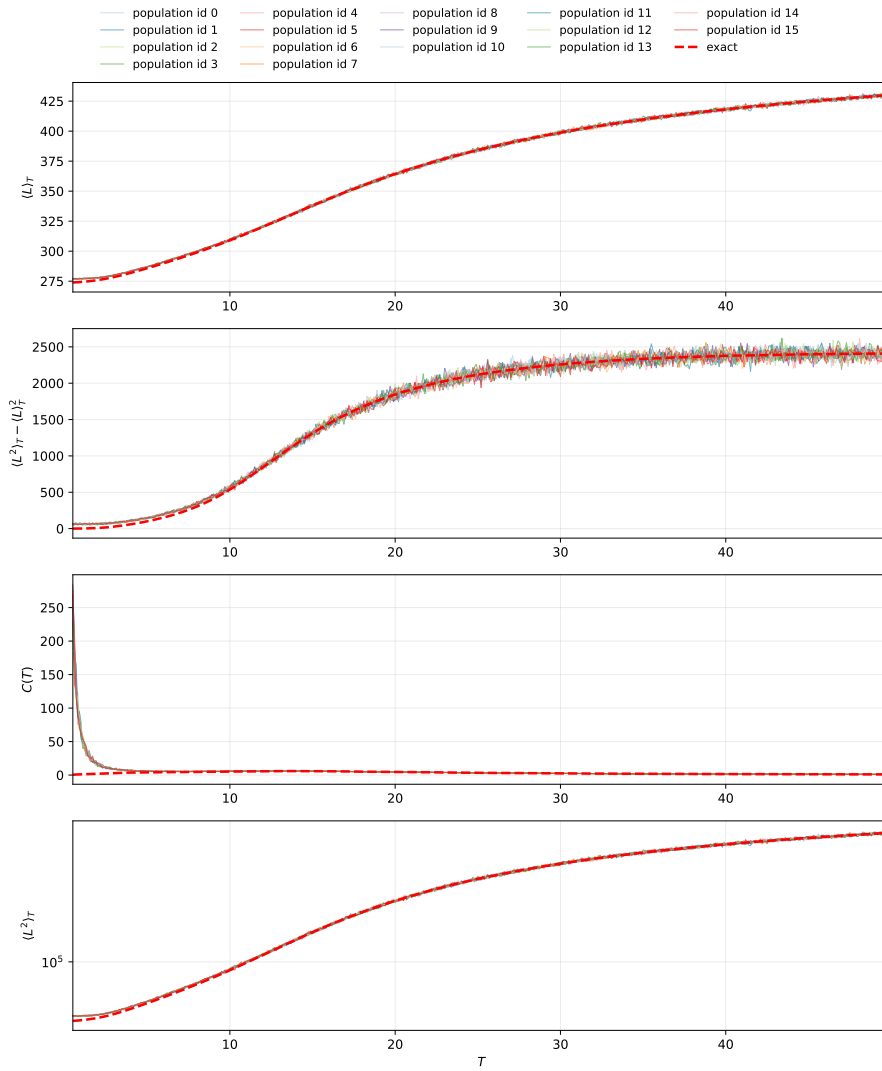


Abbildung 6: Comparison between the exact solution (red) and the sampled observables using the mixed algorithm at N_{Nodes}^2 annealing proposals per sweep

In figure 7, the N_{Nodes}^2 proposals, or "mutations", are restricted every 5th sweep. Now the selection and crossover part dominates for an interval of 5 sweeps, which results in a high mismatch between the exact Boltzmann solution and the generated length means. At low

temperature the mismatch has the same cause as in 6, while at higher temperatures the crossover and selection show the tendency to restrict the routes with higher lengths. Even when the acceptance of the annealing algorithm is between 0.7 and 0.8 at $T=50$, the full N_{Nodes}^2 proposals every 5 sweeps don't allow longer lengths to be generated. The selection and the crossover, on the other hand, seem to produce shorter lengths on average and are, as discussed before, not temperature dependent, as there is no rejection based on the temperature. This is expected and shows the stochastic nature of this optimizer fully.

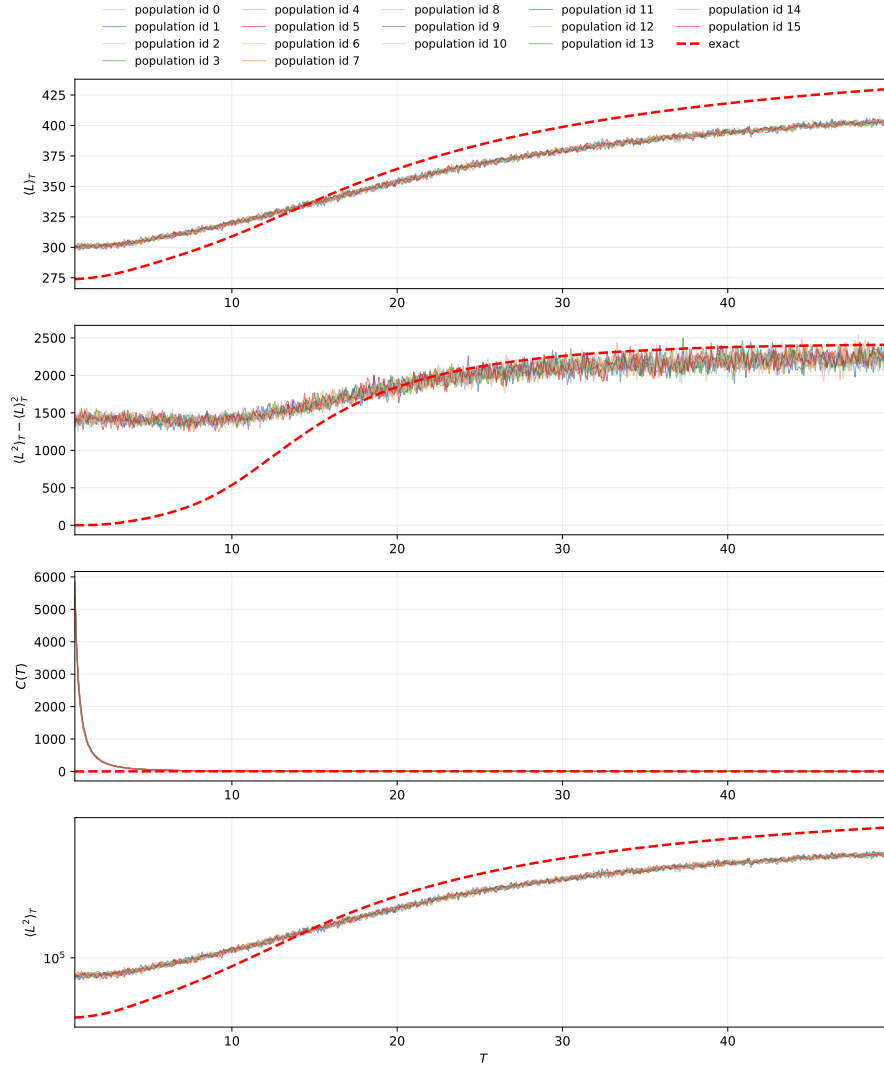


Abbildung 7: Comparison between the exact solution (red) and the sampled observables using the mixed algorithm at N_{Nodes}^2 annealing proposals every 5th sweep

4.2 Autocorrelation

Now this is not really intended to be analyzed in the context of the TSP because it is an optimization problem and the goal is to minimize the observable. It is not necessary to treat the observable as something that is desired to be sampled, like the magnetization of a spin grid in the Ising model, and therefore not needed to estimate its error. Yet this is a Monte Carlo course, and I wanted to play around with error estimators and effective sampling and maybe gain some useful knowledge for this data set. In figure 8 the `ch150` configuration from TSPLIB [1] is shown, which is part of a standardized set of TSP problems. Most of them have a known optimum, which is very useful for benchmarking. This set itself has an optimal minimum length of 6528. The distance metric in this case is

$$L(\tau) = \sum_{k=0}^{N-1} \left[\sqrt{(x_{\tau_k} - x_{\tau_{(k+1) \bmod N}})^2 + (y_{\tau_k} - y_{\tau_{(k+1) \bmod N}})^2} + \frac{1}{2} \right] \quad (22)$$

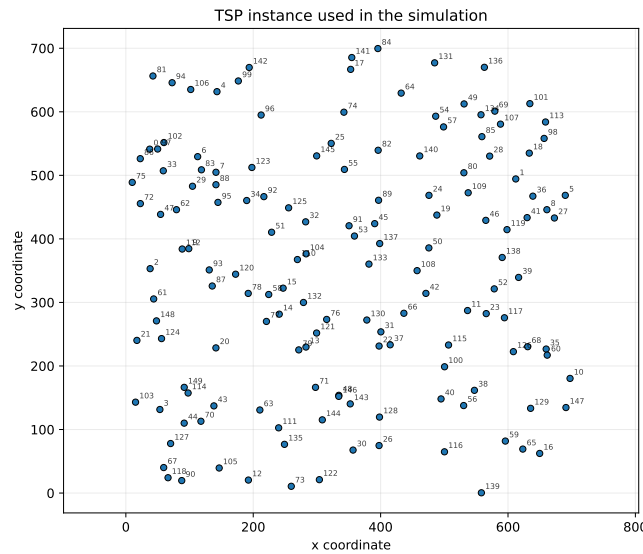


Abbildung 8: `ch150` data set with optimum = 6528

The first step was to estimate the quantities from section 4.1, like the mean tour length, variance, heat capacity, and the acceptance rate. For this, 1500 sweeps were done for thermalization, after which 3000 sweeps were performed at constant temperatures. This was done for both algorithms; however, the combined algorithm was initialized with a population size of 16, for which N^2 annealing/mutation steps were performed each sweep. As the combined algorithm was implemented using parallelization, this only took 2x as long as the annealing algorithm, which was quite surprising. It is again apparent if looking at figure 9 that the combined algorithm doesn't satisfy detailed balance, but this is expected. An interesting feature that was observed is that the best lengths from the population of the combined algorithm (lower left plot: mixed) have a similar peak in the heat capacity as the simulated annealing algorithm. In this case the best length is just the lower boundary of the lengths present in the population. Analyzing the heat capacity further for the simulated annealing one can observe a peak between $T=(30 - 45)$, which is useful information as it tells where the lengths themselves change more rapidly. Therefore, marking a transition between the ordered and disordered states. The acceptance rate doesn't differ for the combined and simulated annealing algorithm, indicating limited

influence of the selection/crossover on the acceptance of proposals. Furthermore, for simulated annealing, the temperature for an uphill acceptance of 0.8 was estimated by proposing changes using the move described in methods and tracking the positive changes. The temperature was estimated to be $T=907$ for this system using:

$$T_0 \approx -\frac{\text{median}(\Delta L_+)}{\ln p_o} \quad (23)$$

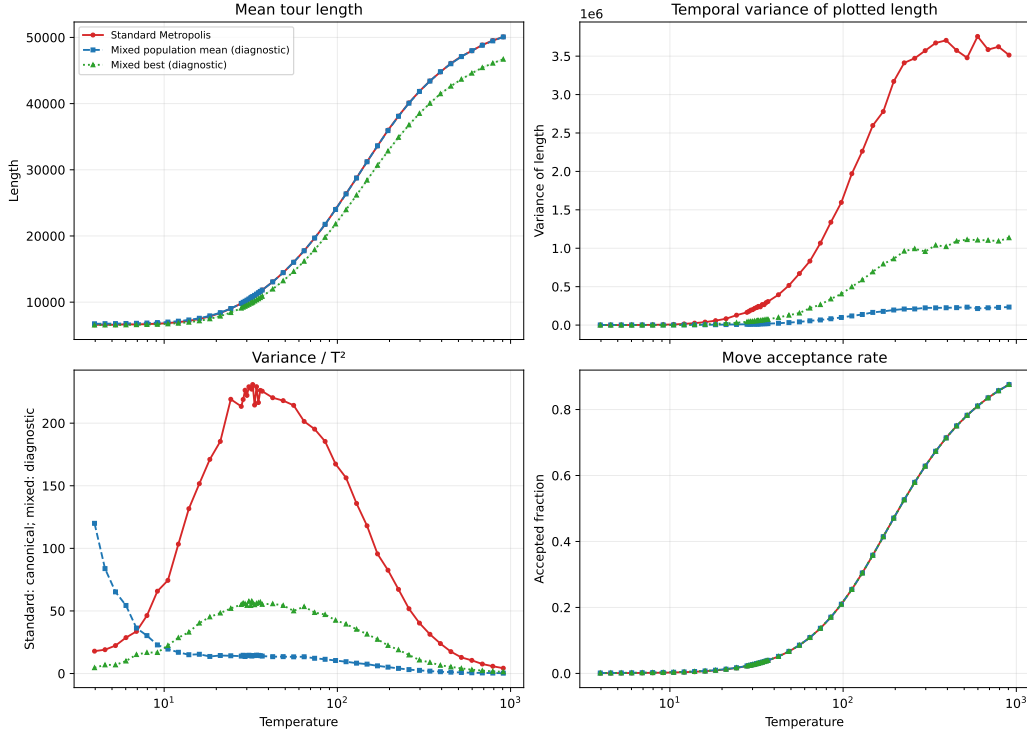


Abbildung 9: Temperature observables tracked for the selected system. Mixed: Combined algorithm , standard metropolis: metropolis-hastings

4.2.1 Metropolis-Hastings Algorithm (Annealing)

The autocorrelation function of the tour length decays over a relatively short number of stored sweeps, but the low-temperature tail and the blocking analysis do not establish a sharp independent-sample regime. The estimated correlation time should therefore be regarded as an observable- and estimator-dependent diagnostic rather than proof that the tours themselves are independent.

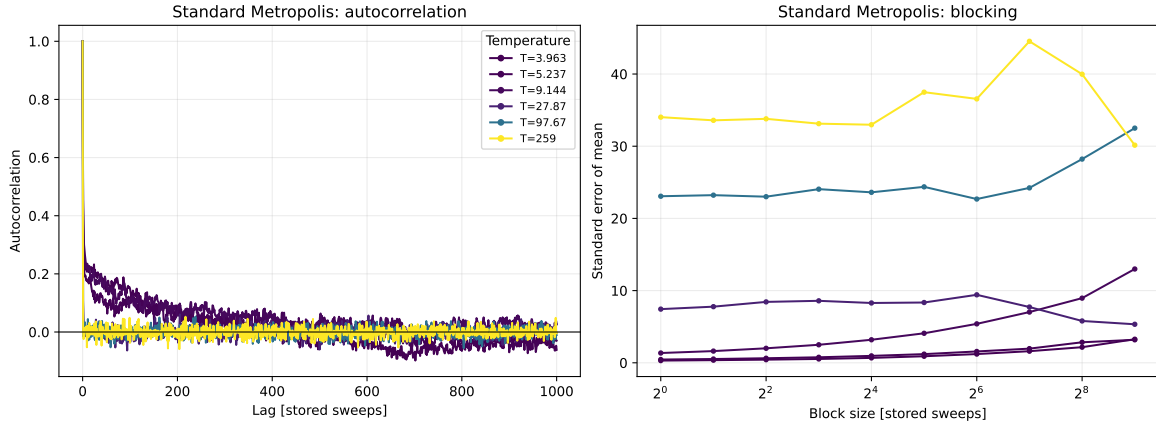


Abbildung 10: autocorrelation function and blocking analysis for selected temperatures of annealing algorithm

4.2.2 Combined Algorithm

For the mixed method, selection and crossover change the population distribution. The mixed population mean and mixed best are not ordinary single-chain observables. Their short ACF means that the population statistic changes rapidly between generations, but it does not imply independent Boltzmann samples.

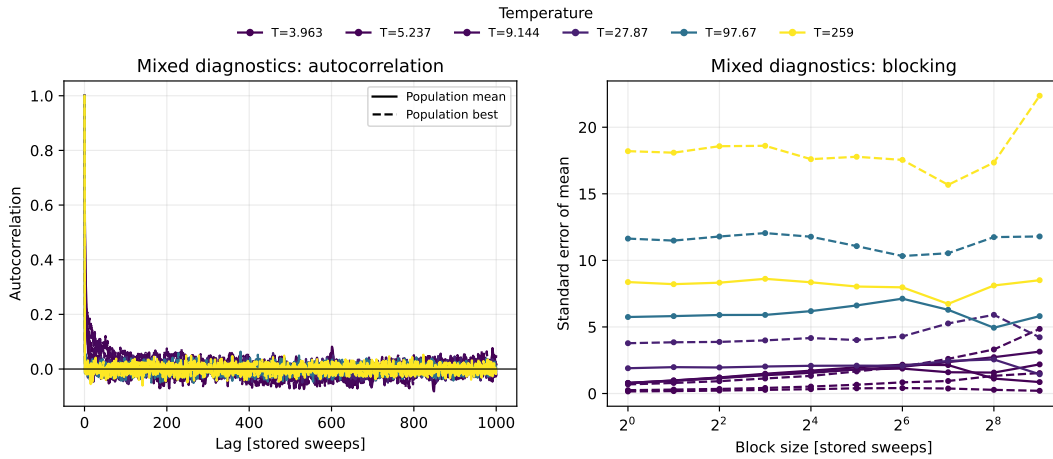


Abbildung 11: autocorrelation function and blocking analysis for selected temperatures of combined algorithm

4.3 Benchmarking

4.3.1 ch150

With the acquired knowledge from the previous section, the temperature curve was selected to start at the calculated $T_0 \approx 1000$. The combined algorithm was initialized using 16 population members and a mutation interval of 1. The temperature curve was selected as follows:

$$T(\text{sweep}) = 1000 \cdot 0.975^{\text{sweep}} \quad (24)$$

To get a fair comparison between the algorithms, the annealing optimization was performed independently 16 times. Out of 16 runs, the annealing algorithm's minimal length was 6528,

which corresponds to the optimal minimum of the data set. The minimal distances out of every run were found and averaged over all runs:

$$\langle L_{\min R, \text{sweeps}} \rangle_R = 6615 \pm 46 \quad (25)$$

while the average sweep count for reaching the minimal distance for a run is:

$$\langle \text{sweeps} \rangle_R = 225 \pm 10 \quad (26)$$

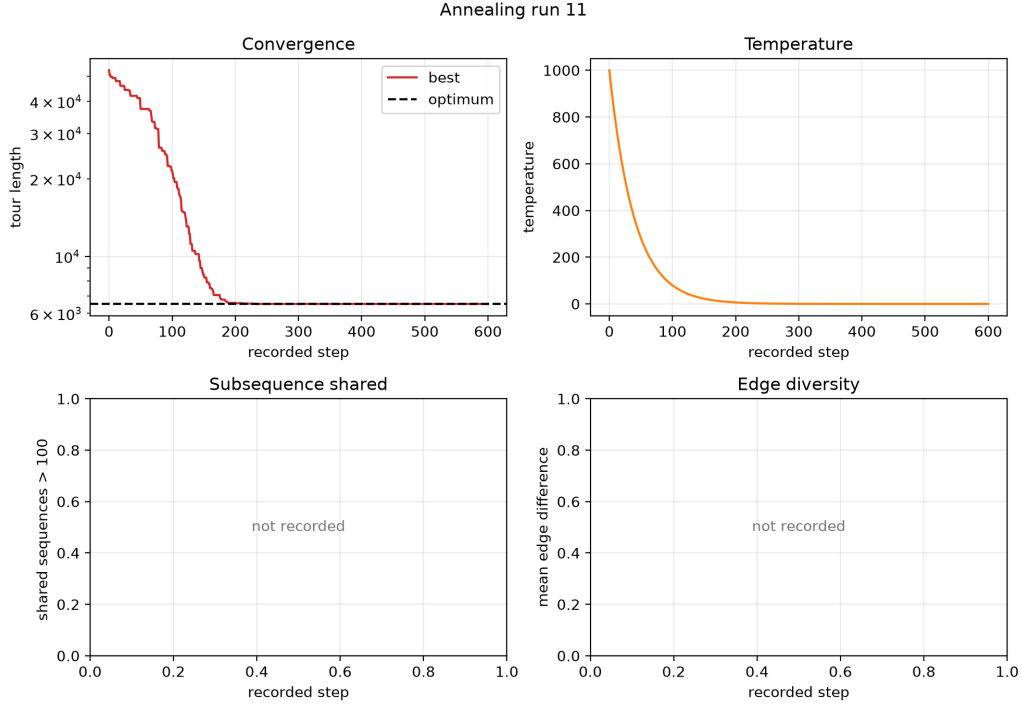


Abbildung 12: Benchmark diagnostics of annealing algorithm for the dataset `ch150`

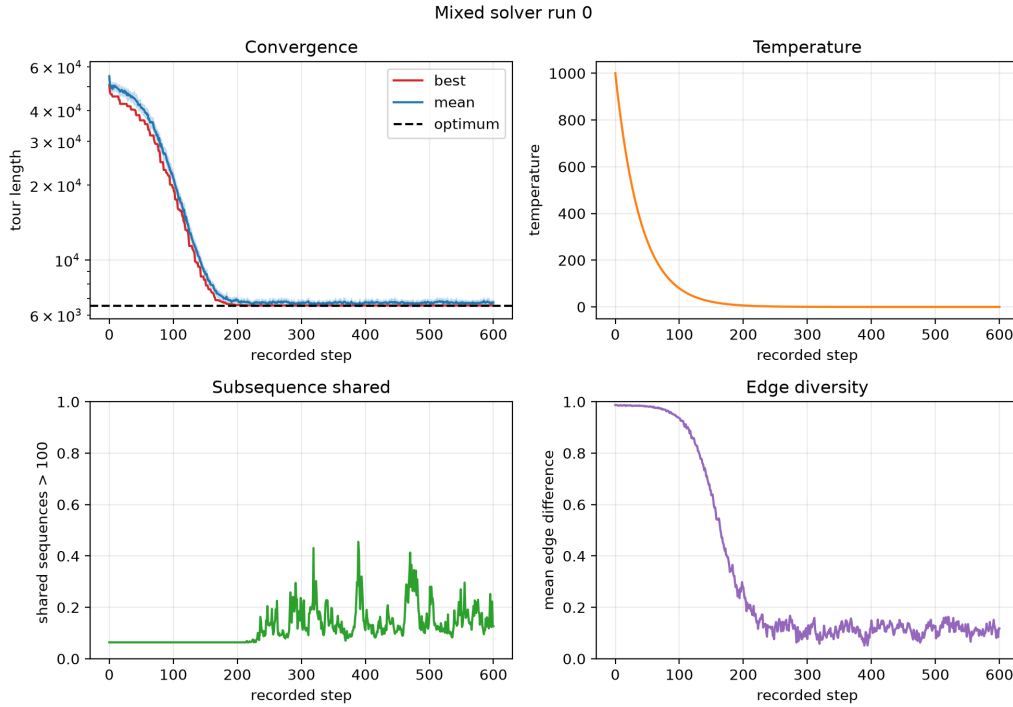
The best tour length reached by the combined algorithm was 6549. The mean of all minimal distances reached by individual population members over all sweeps is:

$$\langle L_{\min P, \text{sweeps}} \rangle_P = 6549 \pm 0 \quad (27)$$

while the average sweep count for reaching the minimal distance for a population member is:

$$\langle \text{sweeps} \rangle_P = 294 \pm 30 \quad (28)$$

The best tour was reached by every population member in a span of 100 sweeps, indicating small diversity.

Abbildung 13: Benchmark diagnostics of combined algorithm for the dataset `ch150`

Looking at Figure 13 the impact of the sudden temperature drop is seen in the edge diversity. At first there are almost no shared edges, which is then reduced, notably with a lag, after the temperature drops. For low temperatures regimes the edge diversity drops quite rapidly and new beneficial mutations due to the low temperature can only contribute restrictively.

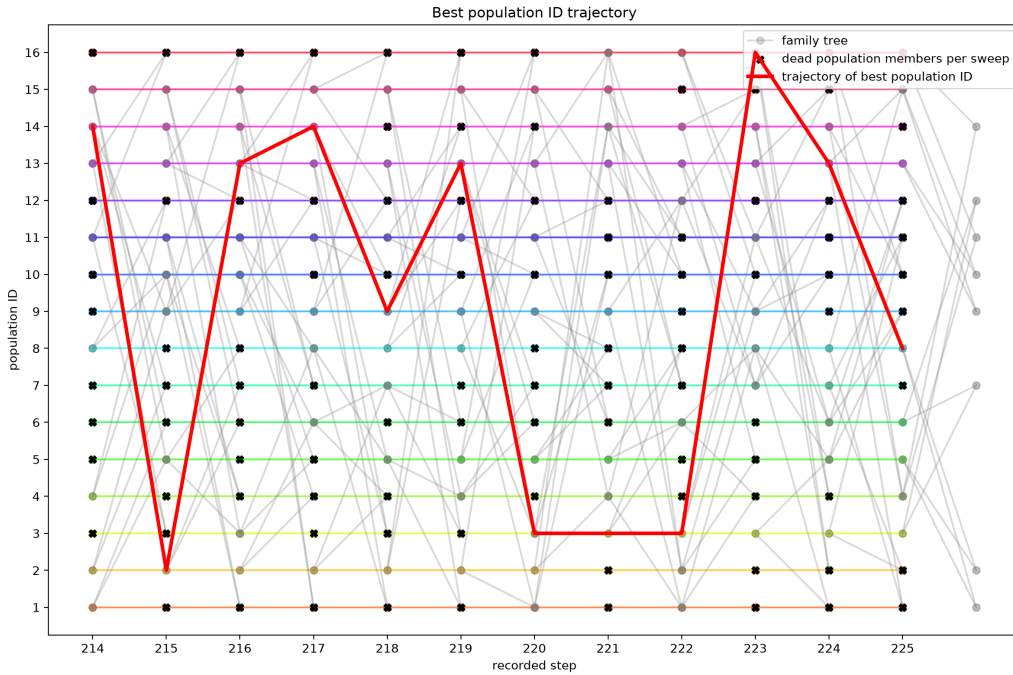


Abbildung 14: Populations family tree in the vicinity of the best tour

When looking at the family tree vicinity of the best found tour in Figure 14 one can observe

that the best population member was obtained after the crossover of two different population member. But after two sweeps this best tour was destroyed through two consecutive mutations.

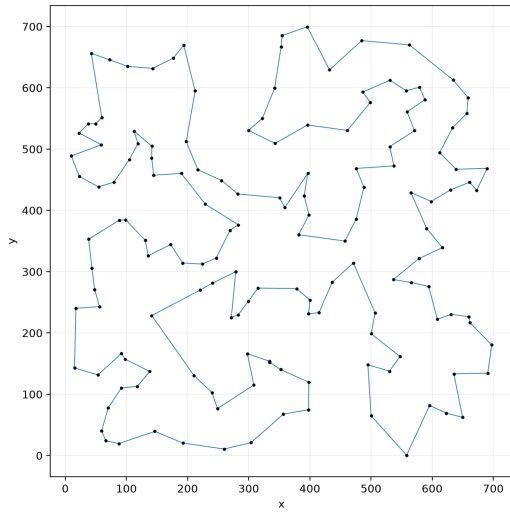


Abbildung 15: Best tour found by simulated annealing

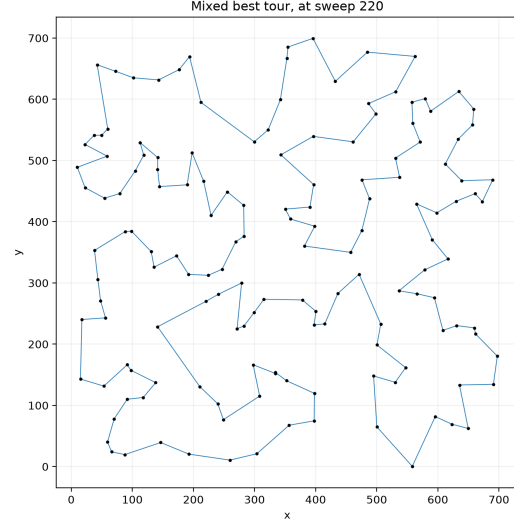


Abbildung 16: Best tour found by combined algorithm

4.3.2 dsj1000 - temperature cycling

Additionally, the `dsj1000` data set was used to benchmark the optimizers. This dataset has 1000 nodes, which makes it computationally more expensive but more interesting to optimize, because the state space is substantially larger. It also gives the opportunity to look more deeply into the population behavior.

First we look at the simulated annealing optimizer. As both optimizers allow for using custom temperature functions, a temperature cycling approach was performed. The temperature function consists of an exponential decay followed by two Gaussian distributions. This should disorganize the current state space vector to allow it to escape from a local minimum.

The combined algorithm was initialized with a population of 20 members and a mutation interval of once every 5th sweep. The mutation itself was done using N_{Nodes}^2 proposals. Again, to get a fair comparison between the two algorithms, the annealing algorithm was run consecutively 20 times, adding the run index to the starting seed to get 20 independent trajectories. Both algorithms were run with 1500 sweeps. Out of 20 runs the annealing algorithms minimal length was 18814218 which it reached at sweep number 946. The minimal distances out of every run were found and averaged over all runs:

$$\langle L_{\min, \text{sweeps}} \rangle_R = 19049045 \pm 103388 \quad (29)$$

while the average sweep count for reaching the minimal distance for a run is:

$$\langle \text{sweeps} \rangle_R = 633 \pm 320 \quad (30)$$

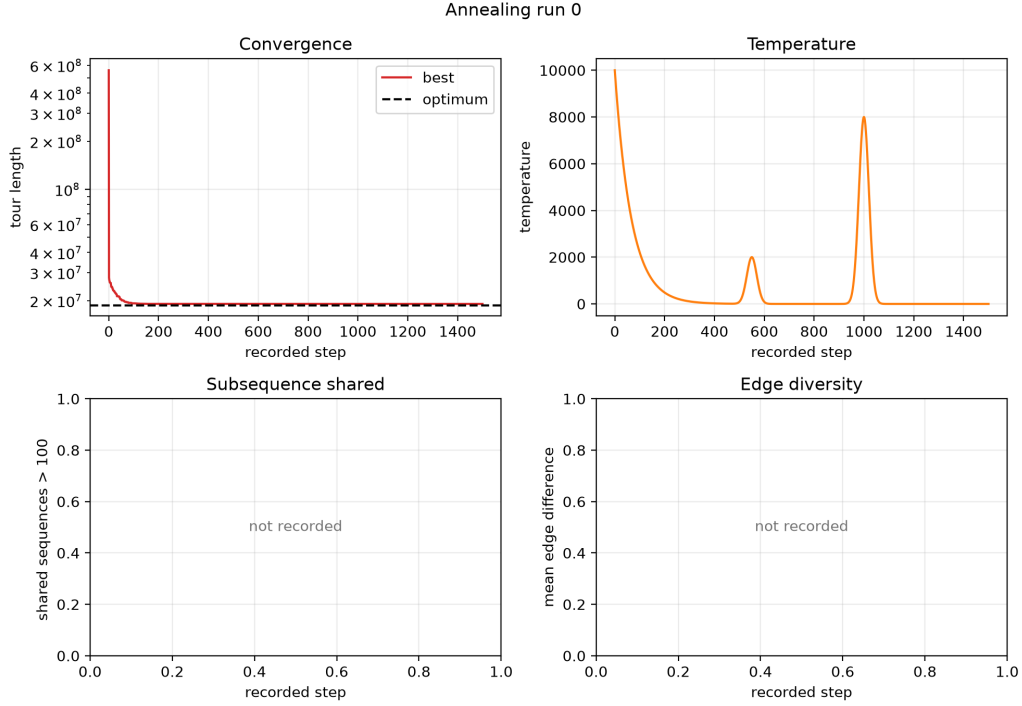


Abbildung 17: Benchmark diagnostics of annealing algorithm for the dataset `dsj1000`

The best tour length reached by the combined algorithm was 18955936. The mean of all minimal distances reached by individual population members over all sweeps is:

$$\langle L_{\min_P, \text{sweeps}} \rangle_P = 18967378 \pm 3967 \quad (31)$$

while the average sweep count for reaching the minimal distance for a population member is:

$$\langle \text{sweeps} \rangle_P = 763 \pm 89 \quad (32)$$

The small standard deviation in the mean of the minimal distances reached as well as the sweep count at which this distance is reached indicates that the population was trapped in a basin consisting of similar local minima. The diversification didn't reach acceptable highs to escape this minima, most likely because the mutation step didn't accept enough uphill proposals. Figure 18 supports this claim as in the region the minima was reached the temperature was low. The Edge diversity was at its low and the amount of shared subsequences larger than 100 was quite high. The basin itself was most likely reached due to the temperature cycling as it gave the population enough room for diversification. The minimal distance of all population members was reached 200 sweeps later, directly before the second cycle.

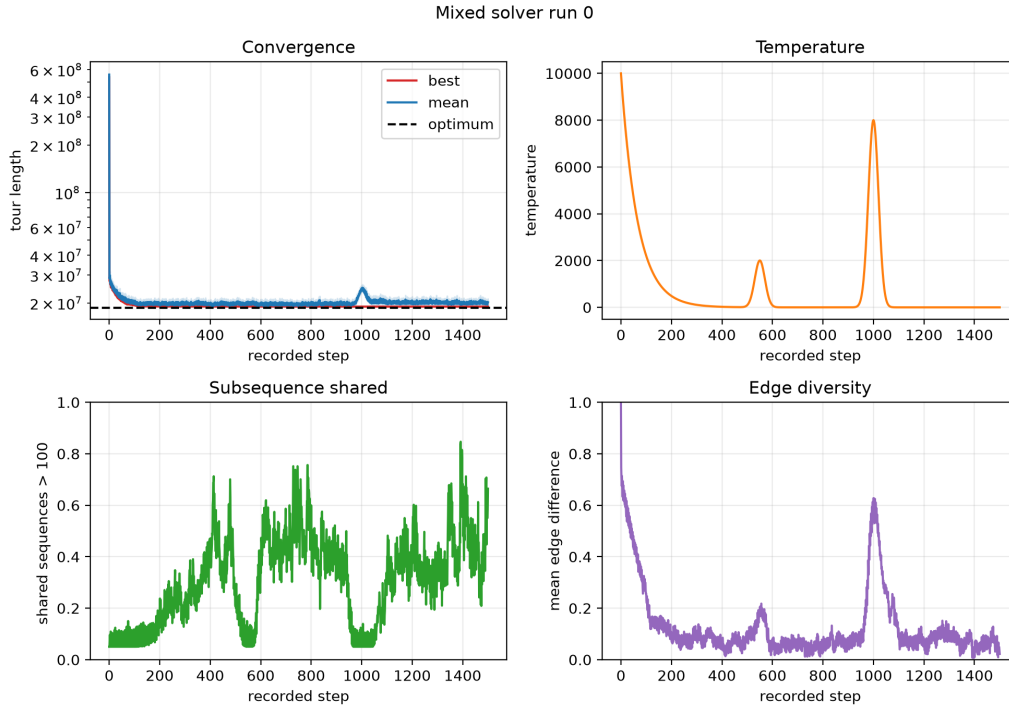


Abbildung 18: Benchmark diagnostics of combined algorithm for the dataset `dsj1000`

When looking at the populations family tree in the region of minimal length an interesting observation can be made. There are two prominent population members, at index 2 and 15 with lengths 18968182 and 18998150 respectively, which evade death over longer sweep counts. These are most likely the part of the best members in the population. At sweep 945, just before the minimal distance was reached both population members mated generating the population member 5. As every 5th sweep there was a mutation step, this member was mutated. This caused the length of this member to decrease to the new population minimum. It seems that the genetic information of the two top 5 population members was beneficial for the metropolis mutation. Before the mutation the crossover of population member 2 ($L_{945} = 18968182$) and population member 15 ($L_{945} = 18998150$) created a tour of length 18968527.

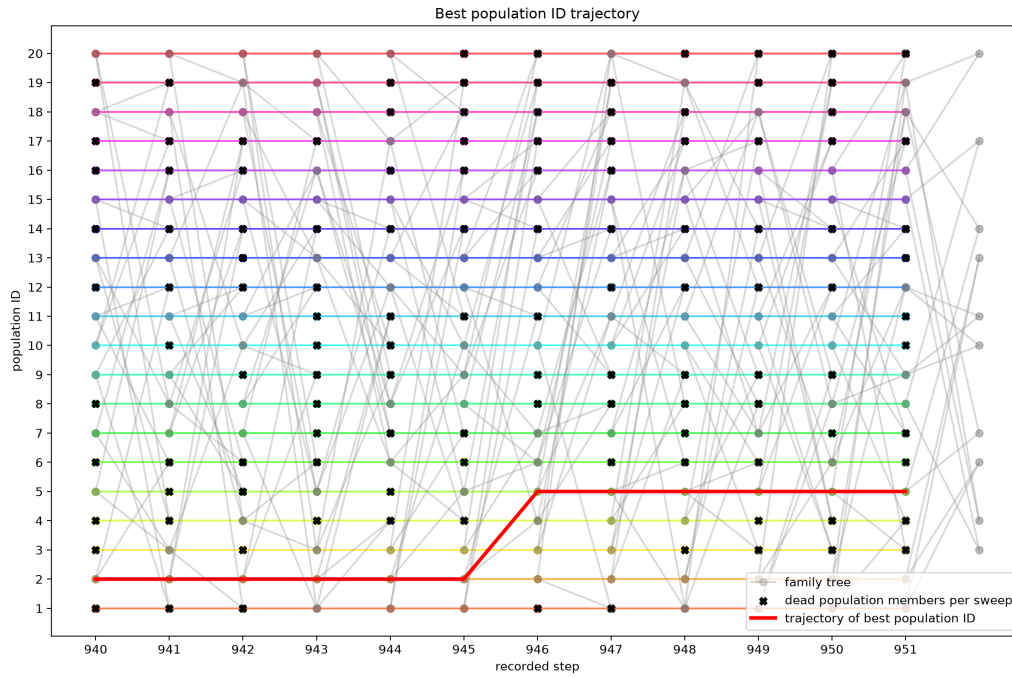
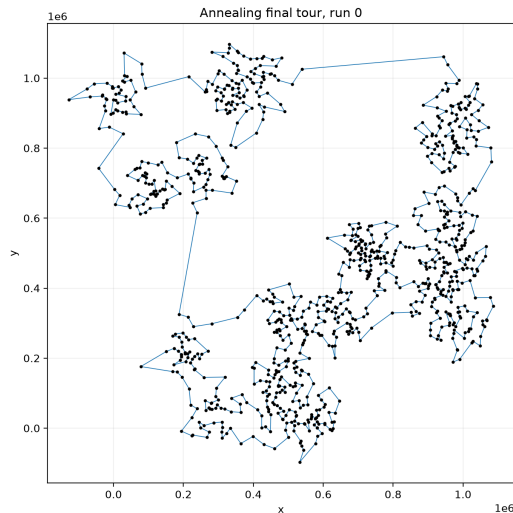
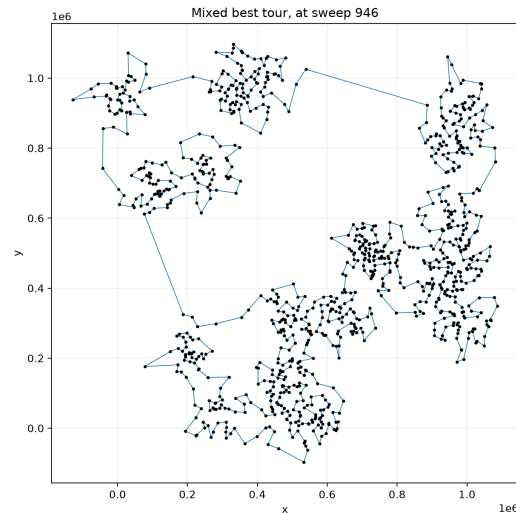


Abbildung 19: Populations family tree in the vicinity of the best tour

In summary the annealing algorithm found a better minimum than the combined algorithm which is approximately 0.7 % lower in length. However the average minimal distance of annealing is higher and has a bigger standart deviation indicating that this was most likely a lucky run. Also the amount of sweeps needed to reach a minimal length varies quite strong. On the other hand the combined algorithm average minimal length spread is lower with an lower average overall. This indicates that the crossover and selection indeed have a positive influence, but a larger dataset with bigger coverage of the parameters aswell as different seeds are needed to confirm the trend.

Abbildung 20: Best tour found by simula-
tion annealing using temper-
ature cyclingAbbildung 21: Best tour found by combi-
ned algorithm using temper-
ature cycling

5 Conclusion

In the benchmarks performed for both algorithms, a clear trend was established. The simulated annealing algorithm performed better for both datasets, which indicates that the combined approach is not beneficial. The crossover and selection showed to have influence on the search for a better tour, as for both datasets the best tour found was generated from a crossover with a consecutive mutation step. Still, it was not enough to beat the annealing algorithm. However, the insight on the behavior of a population and the establishment of a metric for the diversity still constitute a worthy endeavor. The algorithm itself can be improved with better selection rules as well as a less destructive mutation for higher temperatures. Also the crossover part should be rethought and could potentially benefit from different proposal probabilities based on useful genomes. The mean edge diversity could be used for the proposal of crossover pair indices. There are many avenues that can be explored, but this is a task for another day.

Literatur

- [1] UNI HEIDELBERG. Symmetric traveling salesman problems.
<https://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/tspindex.html>